
Testbench for 8-bit ALU

using cocotb verification framework

Submitted to:

Eng. Mohamed El-Adawy

Submitted by:

Menna Assem Fouda



1. RTL Source – alu.v

```
module alu (  
    input wire      clk,  
    input wire      rst,  
    input wire [7:0] a,  
    input wire [7:0] b,  
    input wire [1:0] alu_fun,  
  
    output reg [7:0] alu_out  
);  
  
reg [7:0] alu_out_comb;  
  
always @(posedge clk or negedge rst) begin  
    if (!rst)  
        alu_out <= 8'b0;  
    else  
        alu_out <= alu_out_comb;  
end  
  
always @(*) begin  
    alu_out_comb = 8'b0;  
  
    case (alu_fun)  
        2'b00: alu_out_comb = a + b;  
        2'b01: alu_out_comb = a - b;  
        2'b10: alu_out_comb = a & b;  
        2'b11: alu_out_comb = a | b;  
        default: alu_out_comb = 8'b0;  
    endcase  
end  
  
endmodule
```

2. Makefile

The Makefile configures the simulator, source list, and top-level/testbench module names for cocotb.

```
TOPLEVEL_LANG = verilog  
SIM = icarus  
VERILOG_SOURCES = $(PWD)/alu.v  
TOPLEVEL = alu  
MODULE = test_alu  
include $(shell cocotb-config --makefiles)/Makefile.sim
```

3. Tesy Methodology

4.1 Golden Reference Model

A Python `golden_model()` function reproduces the expected ALU behavior independently of the RTL. Addition and subtraction results are masked to 8 bits (`& 0xff`) to correctly model wraparound/overflow, matching the natural truncation of the hardware's 8-bit register.

```
def golden_model(a, b, op):  
    if op == 0:  
        return (a + b) & 0xff  
    elif op == 1:  
        return (a - b) & 0xff  
    elif op == 2:  
        return a & b  
    else:  
        return a | b
```

4.2 Reset Sequence

`reset_dut()` initializes all inputs to zero, holds reset asserted (`rst = 0`) for two clock edges, then deasserts it (`rst = 1`) and waits one more rising edge before stimulus begins. This guarantees the DUT starts from a known state (`alu_out = 0`) before any test vectors are applied.

```
async def reset_dut(dut):  
    dut.rst.value = 0  
    dut.a.value = 0  
    dut.b.value = 0  
    dut.alu_fun.value = 0  
  
    await RisingEdge(dut.clk)  
    await RisingEdge(dut.clk)  
  
    dut.rst.value = 1  
    await RisingEdge(dut.clk)
```

4.3 Randomized Stimulus

`drive_stimulus()` runs 20 randomized test iterations. In each iteration, operands `a` and `b` are drawn uniformly from 0–255 and the operation `alu_fun` is drawn uniformly from 0–3, covering all four ALU functions. Each vector is applied and held for two rising clock edges to account for the registered output, after which `alu_out` is sampled and compared against the golden model using an assert statement — any mismatch immediately fails the test with the expected vs. actual values reported.

4. Simulation Results

The simulation was run via make, which invoked Icarus Verilog through cocotb. All 20 randomized test vectors, spanning all four operations, produced outputs matching the golden model, and the regression summary reported a full pass with zero failures and zero skips.

```
40.00ns INFO test TEST 0 : A=120 B=23 OP=1 OUT=97
60.00ns INFO test TEST 1 : A=147 B=123 OP=0 OUT=14
80.00ns INFO test TEST 2 : A=134 B=183 OP=1 OUT=207
100.00ns INFO test TEST 3 : A=13 B=201 OP=3 OUT=205
120.00ns INFO test TEST 4 : A=179 B=107 OP=0 OUT=30
140.00ns INFO test TEST 5 : A=236 B=41 OP=1 OUT=195
160.00ns INFO test TEST 6 : A=58 B=81 OP=2 OUT=16
180.00ns INFO test TEST 7 : A=111 B=20 OP=3 OUT=127
200.00ns INFO test TEST 8 : A=15 B=173 OP=2 OUT=13
220.00ns INFO test TEST 9 : A=14 B=165 OP=1 OUT=105
240.00ns INFO test TEST 10 : A=147 B=88 OP=2 OUT=16
260.00ns INFO test TEST 11 : A=101 B=169 OP=2 OUT=33
280.00ns INFO test TEST 12 : A=237 B=74 OP=2 OUT=72
300.00ns INFO test TEST 13 : A=110 B=87 OP=3 OUT=127
320.00ns INFO test TEST 14 : A=148 B=199 OP=2 OUT=132
340.00ns INFO test TEST 15 : A=105 B=194 OP=1 OUT=167
360.00ns INFO test TEST 16 : A=73 B=113 OP=3 OUT=121
380.00ns INFO test TEST 17 : A=110 B=202 OP=1 OUT=164
400.00ns INFO test TEST 18 : A=171 B=64 OP=0 OUT=235
420.00ns INFO test TEST 19 : A=238 B=178 OP=2 OUT=162
420.00ns INFO test All tests passed
420.00ns INFO cocotb.regression ===== TEST FINISHED =====
420.00ns INFO cocotb.regression test_alu.alu_test passed
** TEST STATUS SIM TIME (ns) REAL TIME (s) RATIO (ns/s) **
** test_alu.alu_test PASS 420.00 0.01 68904.31 **
** TESTS=1 PASS=1 FAIL=0 SKIP=0 420.00 0.01 47917.95 **
make[1]: Leaving directory '/home/menna/cocotb_projects/alu_project'
```

Figure 1: cocotb simulation log and regression summary